



ANDE PROJECT

Week 1 — Learning Journal

Adaptive Network Deception Engine
Everything we did, explained in simple terms

Student	Shaun
Week	Week 1 — Foundation & Setup
Date	Monday, July 20 2026
Goal	Set up tools, create project, write first Java code, push to GitHub
Result	SUCCESS — ANDE running and live on GitHub

01. The Tools We Installed

Before writing any code, we needed to set up our workshop. Think of these tools like setting up a kitchen before cooking — you need the right equipment first.

Tool	What it is	Why we need it
Java JDK 25	The Java engine	Your computer cannot understand Java code without this. It translates your code into something the computer can run.
IntelliJ IDEA	Code editor	Where you write all your code. Like Microsoft Word but for Java. It colours your code, catches mistakes, and runs your program.
Maven 3.9.16	Library downloader	Automatically downloads external tools (libraries) for you. Without Maven you'd manually download 20+ files yourself.
Git	Save system	Like checkpoints in a video game. Every time you finish something you save it. If you break something, you go back to the last save.
GitHub	Online storage	Where your Git saves are stored online. Recruiters visit GitHub to see your projects and judge your skills.

02. The Project Structure We Created

We created a folder structure to keep our code organised. Think of it like a filing cabinet — each drawer has a specific purpose.

Folder Structure

```
ande-engine/ ■■■ pom.xml ← Maven shopping list (libraries) ■■■ src/ ■■■ main/ ■■■ java/
■■■ com/ande/ ■■■ Main.java ← WROTE THIS TODAY (entry point) ■■■ network/ ← FUTURE:
server code goes here ■■■ emulator/ ← FUTURE: fake services go here ■■■ intelligence/ ←
FUTURE: attacker profiling goes here ■■■ storage/ ← FUTURE: database code goes here
```

What Each Folder Means

Folder	Simple Explanation
<code>com/ande</code>	Your project's home. Like putting your name on your work. Everything ANDE-related lives inside here.
<code>network/</code>	Will hold the code that listens for attackers connecting. Like a receptionist at a front desk.
<code>emulator/</code>	Will hold code that fakes SSH, FTP, HTTP services. This tricks attackers into thinking they found a real server.
<code>intelligence/</code>	Will hold code that analyses what attackers are doing and scores them. The 'brain' of ANDE.
<code>storage/</code>	Will hold code that saves everything to a database so we can look back at attacker sessions.

03. What pom.xml Does

pom.xml is Maven's shopping list. It tells Maven which external libraries to download for your project. You never download libraries manually — you just list them here and Maven fetches everything automatically.

The libraries we added:

Library 1 — Netty

Netty handles all the complex networking code for us. Building a network server from scratch would take 500+ lines of code. Netty lets us do it in 10 lines. It's what makes ANDE able to accept connections from attackers.

```
io.netty netty-all 4.1.108.Final
```

Library 2 — Logback

Logback is a logging library. Instead of using basic `System.out.println()` to print messages, Logback prints messages with timestamps, colours, and proper formatting. Every line ANDE prints to the screen goes through Logback.

```
ch.qos.logback logback-classic 1.4.14
```

How Maven Works

Step 1: You add a library to pom.xml	Like writing 'Netty' on a shopping list
Step 2: Maven reads the list	Maven checks what you need
Step 3: Maven downloads it automatically	Goes to the internet, fetches the library files
Step 4: Library is ready to use in your code	You can now use Netty in any Java file

04. The Code We Wrote — Main.java

This is the most important file we created today. It is the entry point of ANDE — the first thing Java reads when you start the program. Think of it as the ON button.

Full Code

```
package com.ande; import org.slf4j.Logger; import org.slf4j.LoggerFactory; public class Main
{ // Logger prints messages so we can see what ANDE is doing private static final Logger log
= LoggerFactory.getLogger(Main.class); public static void main(String[] args) {
log.info("====="); log.info(" ANDE - Adaptive Network
Deception Engine"); log.info(" Starting up...");
log.info("====="); log.info("Preparing to listen for
attackers..."); } }
```

Line by Line Explanation

package com.ande;

Tells Java which folder this file belongs to. Every Java file starts with this. Like writing your name on your homework so the teacher knows who wrote it.

import org.slf4j.Logger; import org.slf4j.LoggerFactory;

Brings the Logger tool into this file so we can use it. 'import' is like saying 'bring this tool into my workspace.' Without this, Java would not know what Logger means.

public class Main {

In Java, all code lives inside a 'class'. Think of a class like a container box. Everything we write goes inside this box. 'public' means other parts of the program can see it. 'Main' is the name of this class.

private static final Logger log = LoggerFactory.getLogger(Main.class);

Creates a Logger tool and names it 'log'. private=only this class can use it. static=there is only one copy shared by everything. final=it never changes once created. After this line, whenever we write log.info() it prints a message to the screen.

public static void main(String[] args) {

This is the ON button of the entire program. Every Java program needs exactly one 'main' method. When you run your program, Java looks for this first. public=Java can see it from outside. static=Java can run it without creating an object first. void=it returns nothing.

log.info("...");

Prints a message to the screen with a timestamp. When ANDE ran today, these are the exact lines that appeared showing ANDE starting up successfully.

05. Running ANDE — The First Success

We ran ANDE using Maven in the Command Prompt because IntelliJ had a JDK configuration issue. Maven compiled and ran the code perfectly.

Commands We Used

```
cd C:\Users\Shaun\Desktop\ande-engine
```

Navigate into the project folder. 'cd' means 'change directory' — like double-clicking a folder.

```
mvn compile
```

Tells Maven to compile (translate) all your Java code. If there are mistakes, it tells you here.

```
mvn exec:java  
-Dexec.mainClass="com.ande.Main"
```

Tells Maven to run the Main class. This is what actually starts ANDE.

The Output ANDE Printed

```
13:42:30 [com.ande.Main.main()] INFO com.ande.Main --  
===== 13:42:30 [com.ande.Main.main()] INFO  
com.ande.Main -- ANDE - Adaptive Network Deception Engine 13:42:30 [com.ande.Main.main()]  
INFO com.ande.Main -- Starting up... 13:42:30 [com.ande.Main.main()] INFO com.ande.Main --  
===== 13:42:30 [com.ande.Main.main()] INFO  
com.ande.Main -- Preparing to listen for attackers... [INFO] BUILD SUCCESS
```

What This Output Means

- The timestamp (13:42:30) shows exactly when each line was printed
- com.ande.Main shows which class printed the message
- INFO is the log level — means it is a normal informational message
- BUILD SUCCESS means Maven compiled and ran everything with zero errors
- This was ANDE running for the very first time!

06. Git & GitHub — Saving Your Work

After ANDE ran successfully, we saved all our work to GitHub so it is safely stored online, visible to recruiters, and protected against accidental deletion.

The Simple Analogy

Git Action	Real World Analogy
<code>git init</code>	Setting up a save system in your project for the first time. Like buying a diary to write in.
<code>git add .</code>	Selecting ALL files to include in the next save. Like gathering all your papers before filing them.
<code>git commit -m "..."</code>	Making the actual save with a description. Like sealing an envelope and writing a label on it.
<code>git push</code>	Uploading your save to GitHub online. Like posting that envelope to a safe warehouse.

The Exact Commands We Ran

<code>git init</code>	Created a save system inside the ande-engine folder
<code>git add .</code>	Added all 6 files to the save
<code>git commit -m "Week 1: ANDE first startup working"</code>	Made the save with a description
<code>git branch -M main</code>	Named our main branch 'main'
<code>git remote add origin https://github.com/troubl emaker95/ande-engine.git</code>	Connected local project to GitHub
<code>git push -u origin main</code>	Uploaded everything to GitHub — LIVE on the internet!

Going Forward — Every Session

At the end of every coding session, run these 3 commands to save your work:

```
git add . git commit -m "describe what you built today" git push
```

This keeps your GitHub updated after every session so recruiters always see your latest progress.

07. Week 1 Achievement Summary

■	Java JDK 25 installed	The foundation — your computer can now run Java programs
■	IntelliJ IDEA installed	Your code editor is set up and ready
■	Maven 3.9.16 installed	Library manager working — confirmed with <code>mvn -version</code>
■	ANDE project created	Correct folder structure: <code>src/main/java/com/ande</code>
■	Netty library added	Network handling library downloaded and ready
■	Logback library added	Logging library downloaded and ready
■	Main.java written	First Java file — entry point of ANDE created
■	ANDE ran successfully	BUILD SUCCESS — startup messages printed in terminal
■	GitHub repository created	https://github.com/troublemaker95/ande-engine
■	Code pushed to GitHub	ANDE is live on the internet for the world to see

Week 2 Preview — What's Next

Right now ANDE starts up and prints messages but doesn't do anything else. Next week we make it actually listen for connections — like a real server.

AndeServer.java	Opens a listening port (port 2222) on your computer and waits for anyone to connect. Like a receptionist sitting at a front desk waiting for visitors.
ConnectionHandler.java	When someone connects, this handles what happens. It logs their IP address, what data they send, and when they disconnect.
Testing	Connect to your own tool from Command Prompt using 'telnet localhost 2222' and watch ANDE detect the connection in real-time.

You Built Something Real Today

Most people talk about building cybersecurity tools. Today you actually did it. ANDE is real, it ran, and it is live on GitHub. Keep this momentum going into Week 2.

See you next session. Keep building. ■■